Course Code: CSE2202

Course Title: Algorithms Lab

Section: 4

Submitted By: Mahinur Rahman Mahin

ID: 242014165

Department of Computer Science and Engineering

Submitted To: Md. Masnun, Lecturer,

Department of CSE,

University of Liberal Arts Bangladesh.

Merge Sort:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int mergeSteps , insertionSteps ;
void merge (char arr[], int left, int mid, int right)
{
    mergeSteps++ ;
    int n1 = mid - left + 1 ;
    int n2 = right - mid ;
    char L[n1], R[n2] ;
    for (int i = 0; i < n1; i++)
    {
        mergeSteps++ ;
        L[i] = arr[left + i] ;
    }
    for (int j = 0; j < n2; j++)
    {
        mergeSteps++ ;
        R[j] = arr[mid + 1 + j] ;
    }
    int i = 0, j = 0, k = left ;
    while (i < n1 && j < n2)
    {
        mergeSteps++ ;
        if ((int)L[i] > (int)R[j])
        {
            arr[k++] = L[i++] ;
```

```
        }
        else
        {
            arr[k++] = R[j++] ;
        }
    }
    while (i < n1)
    {
        mergeSteps++ ;
        arr[k++] = L[i++] ;
    }
    while (j < n2)
    {
        mergeSteps++ ;
        arr[k++] = R[j++] ;
    }
}
void mergeSort(char arr[], int left, int right)
{
    mergeSteps++ ;
    if (left < right)
    {
        int mid = (left + right) / 2 ;
        mergeSort(arr, left, mid) ;
        mergeSort(arr, mid + 1, right) ;
        merge(arr, left, mid, right) ;
    }
}
```

```
}
void insertionSort(char arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        insertionSteps++ ;
        char key = arr[i] ;
        int j = i - 1 ;
        while (j >= 0 && (int)arr[j] < (int)key)
        {
            insertionSteps++ ;
            arr[j + 1] = arr[j] ;
            j-- ;
        }
        arr[j + 1] = key ;
    }
}
void printArray(char arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%c ", arr[i]) ;
    }
    printf("\n") ;
}
void copyArray(char source[], char dest[], int n)
{
    for (int i = 0; i < n; i++)
```

```
    for (int i = 0; i < n; i++)
    {
        dest[i] = source[i] ;
    }
}
void reverseArray(char arr[], int n)
{
    int left = 0, right = n - 1 ;
    while (left < right)
    {
        char temp = arr[left] ;
        arr[left] = arr[right] ;
        arr[right] = temp ;
        left++ ;
        right-- ;
    }
}
void testArray(char input[] , int n , char title[])
{
    char arrMerge[n] , arrInsertion[n] ;
    copyArray(input , arrMerge, n) ;
    copyArray(input , arrInsertion, n) ;
    mergeSteps = 0 ;
    insertionSteps = 0 ;
    mergeSort(arrMerge, 0, n - 1) ;
    insertionSort(arrInsertion, n) ;
    printf("\n===== %s =====\n", title) ;
    printf("Original Array: ") ;
```

```
    printf("Original Array: ") ;
    printArray(input, n) ;
    printf ("Merge Sort Result: ") ;
    printArray(arrMerge, n) ;
    printf("Merge Sort Steps: %d\n", mergeSteps) ;
    printf("Insertion Sort Result: ") ;
    printArray(arrInsertion, n) ;
    printf("Insertion Sort Steps: %d\n", insertionSteps) ;
    if (mergeSteps < insertionSteps)
    {
        printf("Best Performance: Merge Sort\n");
    }
    else if (insertionSteps < mergeSteps)
    {
        printf("Best Performance: Insertion Sort\n");
    }
    else
    {
        printf("Both performed equally.\n");
    }
}
int main()
{
    char name[100] ;
    printf("Enter my name: ") ;
    scanf("%s", name) ;
    int n = strlen(name) ;
    char array1[n] ;
```

```c
    printf("Enter my name: ") ;
    scanf("%s", name) ;
    int n = strlen(name) ;
    char array1[n] ;
    char array2[n] ;
    char array3[n] ;
    copyArray(name, array1, n) ;
    copyArray(array1, array2, n) ;
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if ((int)array2[i] > (int)array2[j])
            {
                char temp = array2[i] ;
                array2[i] = array2[j] ;
                array2[j] = temp ;
            }
        }
    }
    copyArray(array2, array3, n) ;
    reverseArray(array3, n) ;
    testArray(array1, n, "Original Name Array") ;
    testArray(array2, n, "Ascending Order Array") ;
    testArray(array3, n, "Descending Order Array") ;
    return 0 ;
}
```

OUTPUT:

```
C:\Users\DELL\Music\Mahin.e

Enter my name: mahinur

===== Original Name Array =====
Original Array: m a h i n u r
Merge Sort Result: u r n m i h a
Merge Sort Steps: 59
Insertion Sort Result: u r n m i h a
Insertion Sort Steps: 23
Best Performance: Insertion Sort

===== Ascending Order Array =====
Original Array: a h i m n r u
Merge Sort Result: u r n m i h a
Merge Sort Steps: 59
Insertion Sort Result: u r n m i h a
Insertion Sort Steps: 27
Best Performance: Insertion Sort

===== Descending Order Array =====
Original Array: u r n m i h a
Merge Sort Result: u r n m i h a
Merge Sort Steps: 59
Insertion Sort Result: u r n m i h a
Insertion Sort Steps: 6
Best Performance: Insertion Sort

Process returned 0 (0x0)   execution time : 4.197 s
Press any key to continue.
```

Quick Sort:

```c
#include <stdio.h>
#include <string.h>
int quickSteps = 0 ;
int mergeSteps = 0 ;
void swap(char *a, char *b)
{
    char temp = *a ;
    *a = *b ;
    *b = temp ;
}
int partition(char arr[], int low, int high)
{
    quickSteps++ ;
    char pivot = arr[high] ;
    int i = low - 1 ;
    for (int j = low; j < high; j++)
    {
        quickSteps++ ;
        if ((int)arr[j] > (int)pivot)
        {
            i++ ;
            swap(&arr[i], &arr[j]) ;
            quickSteps++ ;
        }
    }
    swap(&arr[i + 1], &arr[high]) ;
    quickSteps++ ;
    return i + 1 ;
```

```
    return i + 1 ;
}
void quickSort(char arr[], int low, int high)
{
    quickSteps++ ;
    if (low < high)
    {
        int pi = partition(arr, low, high) ;
        quickSort(arr, low, pi - 1) ;
        quickSort(arr, pi + 1, high) ;
    }
}
void merge(char arr[], int left, int mid, int right)
{
    mergeSteps++ ;
    int n1 = mid - left + 1 ;
    int n2 = right - mid ;
    char L[n1], R[n2] ;
    for (int i = 0; i < n1; i++)
    {
        mergeSteps++ ;
        L[i] = arr[left + i] ;
    }
    for (int j = 0; j < n2; j++)
    {
        mergeSteps++ ;
        R[j] = arr[mid + 1 + j] ;
    }
```

```
55      }
56      int i = 0, j = 0, k = left ;
57      while (i < n1 && j < n2)
58      {
59          mergeSteps++ ;
60          if ((int)L[i] > (int)R[j])
61          {
62              arr[k++] = L[i++] ;
63          }
64          else
65          {
66              arr[k++] = R[j++] ;
67          }
68      }
69      while (i < n1)
70      {
71          mergeSteps++ ;
72          arr[k++] = L[i++] ;
73      }
74      while (j < n2)
75      {
76          mergeSteps++ ;
77          arr[k++] = R[j++] ;
78      }
79  }
80  void mergeSort(char arr[], int left, int right)
81  {
82      mergeSteps++ ;
```

```
81  {
82      mergeSteps++ ;
83      if (left < right)
84      {
85          int mid = (left + right) / 2 ;
86          mergeSort(arr, left, mid) ;
87          mergeSort(arr, mid + 1, right) ;
88          merge(arr, left, mid, right) ;
89      }
90  }
91  void printArray(char arr[], int n)
92  {
93      for (int i = 0; i < n; i++)
94      {
95          printf("%c ", arr[i]) ;
96      }
97      printf("\n");
98  }
99  void copyArray(char source[], char dest[], int n)
100 {
101     for (int i = 0 ; i < n ; i++)
102     {
103         dest[i] = source[i] ;
104     }
105 }
106 void reverseArray(char arr[] , int n)
107 {
108     int l = 0, r = n - 1 ;
```

```
{
    int l = 0, r = n - 1 ;
    while (l < r)
    {
        swap(&arr[l], &arr[r]) ;
        l++ ;
        r-- ;
    }
}
void testArray(char input[], int n, char title[])
{
    char quickArr[n], mergeArr[n] ;
    copyArray(input, quickArr, n) ;
    copyArray(input, mergeArr, n) ;
    quickSteps = 0 ;
    mergeSteps = 0 ;
    quickSort(quickArr, 0, n - 1) ;
    mergeSort(mergeArr, 0, n - 1) ;
    printf("\n===== %s =====\n", title) ;
    printf("Original Array: ") ;
    printArray(input, n) ;
    printf("Quick Sort Result: ") ;
    printArray(quickArr, n) ;
    printf("Quick Sort Steps: %d\n", quickSteps) ;
    printf("Merge Sort Result: ") ;
    printArray(mergeArr, n) ;
    printf("Merge Sort Steps: %d\n", mergeSteps) ;
    if (quickSteps < mergeSteps)
```

```
    printf("Merge Sort Steps: %d\n", mergeSteps) ;
    if (quickSteps < mergeSteps)
    {
        printf("Best Performance: Quick Sort\n") ;
    }
    else if (mergeSteps < quickSteps)
    {
        printf("Best Performance: Merge Sort\n") ;
    }
    else
    {
        printf("Both performed equally.\n") ;
    }
}
int main()
{
    char name[100] ;
    printf("Enter my name : ") ;
    scanf("%s", name) ;
    int n = strlen(name) ;
    char array1[n] ;
    char array2[n] ;
    char array3[n] ;
    copyArray(name, array1, n) ;
    copyArray(array1, array2, n) ;
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
```

```
159     {
160         for (int j = i + 1; j < n; j++)
161         {
162             if ((int)array2[i] > (int)array2[j])
163             {
164                 swap(&array2[i], &array2[j]) ;
165             }
166         }
167     }
168     copyArray(array2, array3, n) ;
169     reverseArray(array3, n) ;
170     testArray(array1, n, "Original Name Array") ;
171     testArray(array2, n, "Ascending Order Array") ;
172     testArray(array3, n, "Descending Order Array") ;
173     return 0 ;
174 }
```

OUTPUT:

```
C:\Users\DELL\Music\Mahin_1

Enter my name : mahinur

===== Original Name Array =====
Original Array: m a h i n u r
Quick Sort Result: u r n m i h a
Quick Sort Steps: 37
Merge Sort Result: u r n m i h a
Merge Sort Steps: 59
Best Performance: Quick Sort

===== Ascending Order Array =====
Original Array: a h i m n r u
Quick Sort Result: u r n m i h a
Quick Sort Steps: 55
Merge Sort Result: u r n m i h a
Merge Sort Steps: 59
Best Performance: Quick Sort

===== Descending Order Array =====
Original Array: u r n m i h a
Quick Sort Result: u r n m i h a
Quick Sort Steps: 67
Merge Sort Result: u r n m i h a
Merge Sort Steps: 59
Best Performance: Merge Sort

Process returned 0 (0x0)   execution time : 4.728 s
Press any key to continue.
```